Glassview: An Orchestrated, Evidence-Driven Architecture for AI-Assisted Software Development

Modern AI-assisted coding environments such as Cursor offer meaningful productivity gains but remain fundamentally constrained by narrow interaction surfaces, temporary project context, and opaque reasoning processes.

OpenGiraffe, in conjunction with the WRCode Developer Kit, introduces a formalized orchestration architecture intended to transform such assistants into verifiable, testable, and extensible components of a broader development ecosystem.

Within this architecture, Glassview is designed as an integrated AI coding analysis and orchestration environment. Its purpose is to operationalize transparent reasoning, multi-agent collaboration, persistent project understanding, and evidence-based software refinement.

Rather than treating individual coding assistants as autonomous authorities, the proposed architecture treats them as specialized components within a controlled development process. Their outputs can be inspected, compared, tested, contextualized, and refined before being accepted into the codebase.

1. From Narrow Interaction Models to Orchestrated Development Processes

Conventional AI coding assistants predominantly operate within a single editor pane and interact with source code through temporary, task-specific context.

Although these systems may inspect files, search repositories, or generate patches, the reasoning process behind their modifications often remains difficult to reconstruct. Relevant architectural decisions, historical constraints, security assumptions, and project-specific intentions may not be available within the current context window.

OpenGiraffe addresses this limitation by expanding the interaction model into coordinated multi-display grids and orchestrated reasoning pipelines.

Through the LetMeGiraffeThatForYou mechanism, developers can elevate a selected output segment, such as a proposed implementation or explanation from Cursor, into a formal Master Tab. This establishes a dedicated analytical workspace in which additional agents and tools can inspect the proposal from different perspectives.

Complementary components orchestrated through Glassview can perform tasks such as:

- repository diff analysis,
- code-integrity validation,
- dependency and impact analysis,
- consistency checking,
- explanation review,
- risk identification,
- test generation,
- and verification of AI-generated modifications.

This interaction paradigm shifts AI assistance away from an isolated point tool and toward a structured, inspectable subsystem within the software-development workflow.

2. Specialized AI Agents and Division of Cognitive Labor

The Glassview environment is designed to support multiple specialized AI agents, each assigned to a distinct analytical or cognitive function within the software-engineering lifecycle.

Possible agent roles include:

Fault-localization agents

These agents identify code regions that are likely to be responsible for persistent defects, unexpected behavior, failed tests, or regressions.

Their task is not necessarily to repair the problem directly, but to reduce the search space and provide evidence for subsequent analysis.

External-knowledge agents

These agents retrieve comparable implementations, relevant design patterns, technical documentation, standards, research, or examples from public repositories and authoritative sources.

Retrieved information remains external evidence rather than automatically accepted project truth.

Cross-reference agents

These agents compare external examples with the local implementation. They can identify architectural similarities, relevant differences, recurring design patterns, security implications, and possible optimizations.

Security and policy agents

Security-oriented agents can inspect proposed modifications for changes to trust boundaries, authorization logic, data handling, execution permissions, sandbox behavior, and other project-specific security constraints.

Testing and verification agents

Verification agents can translate claims about expected behavior into tests, benchmarks, assertions, or other executable checks.

This division of labor enables a distributed reasoning process in which complex tasks are decomposed, independently analyzed, cross-referenced, and reassembled under human oversight.

The objective is not to produce an illusion of consensus through multiple models. Each agent should instead contribute a defined analytical function, evidence source, or verification perspective.

3. Persistent Project Memory Through a Multidimensional Project Wiki

A central planned extension of the architecture is a persistent Project Wiki that can be used across all projects managed by the system.

For general projects, the wiki may contain:

- objectives,
- requirements,
- terminology,
- components,
- workflows,
- decisions,
- constraints,
- milestones,
- risks,

- and historical changes.

For software projects, this wiki can be extended into a machine-readable and human-understandable project memory composed of several connected knowledge dimensions.

3.1 Structural code dimension

The structural dimension represents the observable organization of the codebase.

Depending on the implementation, it may include:

- repositories,
- files,
- modules,
- classes,
- functions,
- interfaces,
- imports,
- function calls,
- dependencies,
- API routes,
- data flows,
- tests,
- configuration elements,
- and version-control relationships.

A code graph can be used to represent these entities and their relationships. This provides agents and developers with a navigable structural view of the project without requiring every task to begin with an unrestricted search through the entire repository.

The structural graph describes what exists and how the relevant code elements are connected.

3.2 Intent and rationale dimension

A parallel rationale or intent dimension describes what the code is intended to accomplish and why a particular implementation or architecture was selected.

Entries may include:

- the purpose of a component,
- expected behavior,
- architectural reasoning,
- implementation assumptions,

- important invariants,
- rejected alternatives,
- known limitations,
- security considerations,
- performance considerations,
- and compatibility requirements.

For example, a function may be structurally connected to an authentication service while its rationale entry explains that authentication logic was intentionally separated from the controller to centralize rate limiting, auditing, and credential handling.

This information is often difficult to infer reliably from code alone.

The rationale layer therefore does not replace the structural graph. It provides a separately maintained knowledge dimension that explains the intention behind the structure.

3.3 Evidence and gap dimension

A selectively applied third dimension may connect relevant architectural or technical decisions with external evidence.

This layer may contain references to:

- scientific publications,
- technical standards,
- established engineering practices,
- comparable implementations,
- benchmarks,
- security guidance,
- and unresolved research questions.

It may also record gaps between the current implementation and an applicable scientific or technical canon.

This dimension should not be applied indiscriminately to every function. It is primarily relevant to components involving security architecture, agent orchestration, retrieval systems, sandbox isolation, cryptographic mechanisms, evaluation methods, or other technically significant decisions.

A gap entry might indicate that a particular design is supported indirectly by existing research but has not yet been evaluated for the specific repository structure, execution environment, or threat model used by the project.

3.4 Shared identity and provenance

The different knowledge dimensions should remain logically distinct while referring to common project entities through stable identifiers.

A code symbol, architectural decision, test, rationale entry, risk, and scientific reference may therefore be linked without being stored as one undifferentiated body of information.

Each knowledge item may additionally include provenance data such as:

- whether it was written by a developer or generated by an AI,
- which model or agent produced it,
- when it was created,
- which code revision it refers to,
- whether it has been reviewed,
- and whether the underlying code has subsequently changed.

This allows the system to distinguish verified project knowledge from provisional AI-generated interpretations.

3.5 Detecting outdated project knowledge

Persistent documentation becomes dangerous when it remains accessible after the underlying implementation has changed.

The project-memory architecture should therefore support freshness and contradiction states.

When a relevant code entity changes, associated rationale entries, explanations, test assumptions, or evidence mappings may be marked as potentially outdated.

Possible states include:

- generated,
- review required,
- verified,
- stale,
- contradicted,
- superseded,
- and historical.

The system should not silently overwrite verified explanations. Instead, it may generate a proposed revision and display the difference between the previous rationale and the newly inferred interpretation.

3.6 Local and cloud-based maintenance

The project memory may be maintained through human input, local AI models, cloud-based AI services, or combinations of these methods.

Local models may be preferred for sensitive repositories or routine documentation tasks. Cloud models may be used where permitted for more complex reasoning or external research.

Regardless of the selected model, AI-generated entries should retain their provenance and verification state.

The purpose is not to automate documentation without control. It is to make project knowledge continuously maintainable while preserving visibility into how each entry was produced.

4. Project-Aware Context Retrieval for Coding Agents

Coding agents currently spend substantial effort reconstructing project context from source files, repository searches, configuration files, and temporary conversation history.

The proposed Project Wiki and graph architecture enables a different retrieval sequence.

Before broadly searching the entire codebase, an agent may request a task-specific project context package containing:

- relevant project objectives,
- affected components,
- connected code entities,
- architectural decisions,
- applicable invariants,
- known risks,
- relevant tests,
- recent changes,
- and unresolved gaps.

The agent may then inspect the identified source-code regions directly and expand its search only where the supplied context is insufficient.

This mechanism does not prevent direct code inspection. The source code remains the authoritative representation of the implementation.

Instead, project-aware retrieval provides an initial orientation layer that helps agents understand which code is likely to matter and which project-specific constraints should guide the task.

A possible interaction sequence is:

1. The developer or coding agent submits a task.
2. The project-memory interface identifies relevant concepts and components.
3. The code graph returns related symbols, dependencies, tests, and change history.
4. The rationale dimension supplies purposes, architectural decisions, and invariants.
5. The evidence layer supplies relevant standards, research, or documented gaps where applicable.
6. A compact context package is assembled according to an available token or information budget.

7. The coding agent inspects the relevant source-code sections and performs the requested task.
8. Glassview evaluates the resulting modification against the same project context.

This creates a shared understanding layer between developers, coding assistants, and verification agents.

The interface may be exposed through a dedicated API or an agent-compatible protocol, allowing different coding environments to access the same persistent project knowledge.

5. Automatic Prompt Engineering and Iterative DeepFix Workflows

A defining capability of the framework is its automated prompt-engineering layer.

Glassview aggregates selected findings from multiple agents and project-memory dimensions, synthesizing them into structured prompts or context packages for downstream coding assistants such as Cursor.

This can reduce the cognitive burden associated with manually composing large, multi-step instructions.

Instead of forwarding every available document or repository file, Glassview can construct prompts around the specific task and its verified dependencies.

A generated prompt may include:

- the requested modification,
- relevant source locations,
- affected architectural components,
- project-specific constraints,
- invariants that must remain valid,
- applicable tests,
- previous failed approaches,
- and unresolved questions.

Glassview may further incorporate a DeepFix-like iterative optimization cycle.

Diagnostic artifacts such as:

- logs,
- console output,
- execution traces,
- compiler errors,
- static-analysis findings,
- failed tests,

- and runtime anomalies

are not treated as inert byproducts.

They are transformed into structured feedback signals and connected to the relevant project entities. These signals can refine subsequent prompts, allowing the orchestrator to iteratively converge toward a better-supported solution.

Project memory improves this process by preventing each iteration from beginning without historical context. Failed approaches, discovered constraints, and accepted explanations can remain available to subsequent agents and development sessions.

6. Transforming AI Explanations Into Executable Test Specifications

A central design objective of the system is to operationalize AI explanations as engineering artifacts.

Rather than treating explanations as disposable descriptive text, Glassview can interpret them as provisional claims about intended code behavior.

These claims may then be translated into:

- test cases,
- assertions,
- benchmarks,
- static-analysis rules,
- validation procedures,
- or observable acceptance conditions.

For example, if an AI explanation states that a function validates user credentials, Glassview can derive test cases involving valid credentials, invalid credentials, malformed input, repeated attempts, and relevant edge conditions.

If an explanation claims that a change improves performance, Glassview can benchmark the affected operation against an established baseline.

If an explanation states that a security boundary remains intact, the system can generate or select tests intended to challenge that boundary.

The resulting tests do not automatically prove that the explanation is complete or correct. They provide executable evidence for or against the specific claims made by the AI.

Verified explanations may subsequently contribute to the rationale dimension of the Project Wiki. Explanations contradicted by test evidence can be rejected, revised, or retained as historical records of an unsuccessful approach.

This establishes a feedback relationship between:

- natural-language reasoning,

- intended behavior,
- actual code,
- executable tests,
- and persistent project knowledge.

7. Automated Testing, Risk Propagation, and Repository Stewardship

Glassview connects its reasoning process with automated testing and workflow systems, potentially including browser-testing frameworks, local test runners, CI pipelines, and orchestration tools such as n8n.

A proposed development cycle may include:

1. AI explanations and generated code are decomposed into claims and expected behaviors.
2. Relevant tests are selected or generated.
3. Tests are executed across normal, boundary, and failure conditions.
4. Failures and anomalies are returned to the reasoning layer.
5. Console output, logs, traces, and test results are merged into a unified diagnostic context.
6. The code graph identifies directly and indirectly affected components.
7. The rationale graph identifies project decisions or invariants that may be affected.
8. Risk analysis determines whether additional regression or security testing is required.
9. A revised implementation is generated or requested.
10. The result is presented for human inspection and approval.

When a modification has been accepted, the system may propose updates to:

- project documentation,
- rationale entries,
- architectural decisions,
- tests,
- risk records,
- change history,
- and evidence mappings.

These updates should remain reviewable and version-controlled.

Glassview may additionally propose or execute version-controlled backups according to the permissions granted by the developer. Commit hashes, signatures, timestamps, test evidence, and analysis artifacts can contribute to an auditable development history.

8. Glassview as an Explainable Change-Intelligence Layer

Within the broader architecture, Glassview serves not only as an orchestration engine but also as an explainable change-intelligence layer.

When a code modification is detected, Glassview can map the changed lines to the affected project entities and retrieve the corresponding knowledge from the Project Wiki.

A change report may therefore include:

- modified files and symbols,
- direct and indirect dependencies,
- potentially affected behavior,
- relevant architectural decisions,
- applicable invariants,
- associated tests,
- security implications,
- known technical risks,
- stale project knowledge,
- and unresolved evidence gaps.

This report allows a developer to inspect not only what changed, but why the change matters within the broader project.

For example, removing an audit-logging call may appear locally harmless. Through the rationale layer, Glassview may discover that complete authentication-event auditing is a documented security invariant. Through the structural graph, it may identify the affected authentication paths. Through the test layer, it may determine that no current integration test verifies the invariant.

The resulting analysis can therefore communicate a traceable chain:

- which code changed,
- which project rule may have been violated,
- what supporting evidence exists,
- which tests were executed,
- and what remains uncertain.

This is fundamentally different from presenting a binary approval score or an unexplained AI recommendation.

9. Continuous Project Learning

The Project Wiki and Glassview can form a controlled feedback loop.

The Project Wiki supplies context for change analysis. Glassview uses that context to evaluate modifications. Accepted changes, verified explanations, new tests, and discovered constraints can then be proposed as updates to the project memory.

The process may be summarized as:

Project knowledge informs code changes.

Code changes trigger contextual analysis.

Analysis generates evidence and proposed knowledge updates.

Human-reviewed updates improve future project understanding.

This creates a persistent engineering memory that can develop alongside the codebase without treating AI-generated interpretations as automatically authoritative.

Over time, the system may help reduce repeated rediscovery of:

- architectural reasoning,
- previously encountered failure modes,
- security assumptions,
- rejected implementation strategies,
- dependency relationships,
- and project-specific development conventions.

10. Transparent, Evidence-Driven, and Augmented Development

The resulting architecture promotes a transparent development methodology.

Developers are not limited to determining whether a generated solution appears to work. They can inspect the reasoning and evidence associated with each iteration:

- agent explanations,
- project context,
- code changes,
- architectural relationships,
- executed tests,
- diagnostic output,
- risk-propagation analysis,
- scientific or technical references,
- and unresolved uncertainty.

Glassview is intended to present this information at different levels of granularity.

A developer may use a concise summary for rapid iteration or inspect the complete analytical chain for high-assurance changes.

By combining:

- project-aware context retrieval,
- persistent multidimensional project memory,
- specialized agent collaboration,
- explanation-derived testing,
- structured prompt generation,
- risk-aware refinement,
- and auditable repository evolution,

OpenGiraffe and Glassview establish a conceptual foundation for augmented software engineering.

The proposed architecture moves beyond the use of AI as an isolated code generator. It treats AI systems as inspectable contributors within a broader development process in which claims can be challenged, changes can be traced, project intentions can persist, and human developers retain control over acceptance and execution.

11. Development Status

The architecture described in this article contains both existing development concepts and planned future capabilities.

The multidimensional Project Wiki, persistent rationale layer, scientific evidence mapping, and project-aware agent interface represent roadmap components whose final implementation and technical validation remain subject to further development and testing.

The purpose of this publication is to document the intended architecture, its relationships, and its potential development direction. It should not be interpreted as evidence that every described component has already been implemented, validated, or released.

Persistent Background Optimization and Long-Horizon Project Analysis

Glassview is not limited to reactive analysis triggered by an individual code modification or developer request. A planned capability of the architecture is the execution of extended analytical sessions in which local or cloud-based AI systems examine a project over longer periods, including scheduled overnight runs and periods of otherwise unused computational capacity.

This long-horizon operating mode allows complex optimization and assurance tasks to be decomposed into smaller analytical jobs rather than requiring a single model to produce an immediate and complete answer.

Possible background tasks include:

- architectural consistency analysis,

- security and trust-boundary review,
- dependency-risk analysis,
- identification of insufficiently tested components,
- detection of outdated rationale entries,
- comparison between documented intent and actual implementation,
- technical-debt discovery,
- performance-optimization analysis,
- scientific and technical gap analysis,
- review of unresolved development questions,
- and generation of proposed updates to the Project Wiki.

Local and cloud-based execution

Background analyses may be executed through local AI models, external cloud models, or a controlled combination of both.

Local models may process private source code, internal documentation, logs, architectural information, and other sensitive project data without transmitting them to an external provider. Cloud-based systems may optionally be used for tasks requiring stronger general reasoning, broader external research, or access to specialized capabilities.

The execution strategy can be selected according to project policy, data classification, model capability, available hardware, cost, and required assurance level.

A hybrid workflow may assign sensitive repository analysis to local models while allowing a cloud-based research agent to examine public standards, scientific literature, or publicly available reference implementations. Only the necessary and permitted information should be exchanged between these environments.

Task decomposition and persistent analysis queues

Long-running optimization should not be treated as one unstructured model conversation.

Glassview may instead maintain a persistent analysis queue containing defined tasks such as:

- examine authentication-related trust boundaries,
- identify architectural drift within a selected module,
- compare implemented behavior with documented invariants,
- locate insufficiently justified dependencies,
- evaluate test coverage for high-risk execution paths,
- or investigate an unresolved scientific or engineering gap.

Each task can be divided into smaller units and assigned to specialized agents. Intermediate findings can be stored as structured artifacts, allowing subsequent agents or later sessions to continue the analysis without reconstructing the entire reasoning context.

This enables weaker or smaller local models to contribute useful work through repeated, constrained, and independently verifiable steps.

Evidence accumulation rather than unrestricted autonomy

The purpose of background optimization is not to permit an AI system to modify a project invisibly or without control.

By default, background agents should operate on a defined repository snapshot and produce evidence, findings, hypotheses, and proposed modifications.

Their outputs may include:

- identified risks,
- supporting code references,
- graph paths,
- test proposals,
- suggested rationale revisions,
- architectural alternatives,
- confidence assessments,
- unresolved questions,
- and candidate code changes.

These artifacts can subsequently be reviewed through Glassview.

Verified project knowledge should not be silently replaced by newly generated interpretations. When a background agent reaches a conclusion that conflicts with an existing architectural decision, invariant, or rationale entry, the conflict should be presented explicitly.

Resource and scope governance

Each background session may be governed through a formal execution policy defining:

- the permitted repositories and branches,
- accessible project-memory dimensions,
- allowed tools,
- local or external model providers,
- maximum execution duration,
- compute and token budgets,
- network-access permissions,

- data-export restrictions,
- and whether code modifications may be proposed or executed.

High-assurance projects may restrict background agents to read-only analysis. Other environments may allow them to create isolated branches, patches, tests, or documentation proposals while still requiring human approval before integration.

Continuous gap analysis

The evidence and gap dimension of the Project Wiki can provide a persistent work surface for long-term research and optimization.

A gap may represent:

- an architectural assumption that has not been validated,
- a security claim without a corresponding test,
- an implementation that differs from an applicable standard,
- missing empirical evidence,
- an unresolved performance question,
- an insufficiently documented design decision,
- or a relevant scientific method that has not yet been evaluated for the project.

Background agents can periodically revisit such gaps as the codebase, available models, external research, and project requirements evolve.

The system may update the status of a gap from unidentified to under investigation, partially supported, contradicted, resolved, or requiring human judgment.

Background security analysis

Security analysis benefits particularly from extended execution because many risks are not visible within an isolated code fragment.

A long-running security workflow may traverse the code graph, inspect trust boundaries, follow data flows, compare permission checks across execution paths, analyze dependencies, and connect findings to documented security invariants.

Multiple specialized agents may independently examine the same area from different perspectives, such as:

- authorization,
- data exposure,
- injection risks,
- sandbox escape paths,
- supply-chain dependencies,
- secret handling,

- unsafe tool execution,
- or unintended network communication.

Glassview can present the resulting findings together with their evidence, disagreements, confidence levels, and affected project entities.

Integration with the development workflow

A typical overnight workflow may proceed as follows:

1. A repository commit or snapshot is selected as the analysis baseline.
2. Glassview creates a queue of predefined or automatically identified analysis tasks.
3. Local and permitted cloud agents execute the tasks within defined resource and access policies.
4. Findings are mapped to code entities, rationale entries, tests, risks, and evidence records.
5. Conflicting conclusions are retained rather than artificially merged into a false consensus.
6. Candidate tests, patches, documentation updates, and new gap records are generated.
7. The results are assembled into a reviewable Glassview report.
8. A developer decides which findings should be rejected, investigated, implemented, or incorporated into the persistent Project Wiki.

This enables useful optimization work to continue outside the immediate interactive coding session while preserving human control over the codebase and its authoritative project knowledge.

The intended result is a system that does not merely respond to development activity but continuously prepares better evidence, context, and improvement proposals for future development decisions.